Task Discovery Service (TDS): A Dual-Architecture Service Registry for Distributed Systems

Final Year Project Report

February 2026

1 Introduction and Motivation

In large distributed systems, static service routing can become a serious scaling bottleneck. In the system I worked on at Cubic Transportation Systems, the issue was not how requests were received but how they were forwarded: output endpoints were fixed in code. The component could accept requests from a changing number of devices, yet every routing change required redevelopment and full regression testing. That made scaling slow and expensive. As demand grew, the original resolvers were replaced with load balancers because the fixed design no longer matched operational needs. This project grew from that bottleneck and examined how service discovery could be made more dynamic and easier to scale.

1.1 Problem Statement

The problem had two main parts:

1. The component remained static even though it performed the same routing role for each request from the devices below it.
2. The server became overloaded because it mainly accepted data from the requesting device and forwarded it to the resolver. The component did not process or inspect the payload itself.

I addressed these issues by designing an architecture that could scale with the system while remaining independent of the content of the data being sent. The focus was efficient discovery and routing rather than payload handling.

1.2 Research Objectives

This project addresses the service discovery problem by developing the Task Discovery Service (TDS), a flexible registry system that supports both centralised and peer-to-peer (P2P) operational modes within a unified architecture. The primary objectives are:

1. Design and implement a dual-mode service registry supporting both centralised and P2P architectures
2. Develop efficient algorithms for service registration, query routing, and load balancing
3. Create a comprehensive monitoring interface for real-time system visualisation
4. Evaluate performance characteristics under various network conditions and load patterns
5. Provide multiple transport and security options including UDP, TCP, and mutual TLS authentication

1.3 Contribution

The main contribution of this final-year project is the design and implementation of TDS, a service-discovery system that presents a single client-facing interface while supporting both centralised and peer-to-peer backends. Services use the same registration and lookup workflow in either mode, allowing the system to be deployed with either a central registry or a DHT-based peer network without changing the client interaction model.

The project also adds a practical set of supporting features around that core design. These include multiple transport options, optional TLS and mutual TLS for the centralised path, firewall-aware query filtering, live monitoring through a terminal dashboard, and a transit-system simulation based on a highly simplified version of the TfL Oyster, payment card, and physical ticket system. Together, these elements make the project a complete implementation of a flexible service-discovery architecture for the problem that motivated the work.

2 Project Background and Context

2.1 Service Discovery in Distributed Systems

Service discovery allows one part of a distributed system to find another at runtime, avoiding hard-coded addresses in an environment where instances are often restarted, scaled, or replaced. Most solutions fall into three groups.

2.1.1 Centralised Registry Approaches

Centralised approaches keep a single main registry. Systems such as etcd use Raft consensus to provide strongly consistent control-plane state [3, 9, 16]. Kubernetes builds on this by keeping stable service identities while EndpointSlices track changing pods [12]; Consul follows a similar pattern with added health checks [2, 15]. The main strength is simplicity and easy observation. The main weakness is that the registry becomes a dependency, so its failure disrupts discovery.

2.1.2 Decentralised Approaches

Decentralised approaches spread lookup data across peers. Chord and Kademlia showed that hashing keys to node IDs allows efficient $O(\log n)$ routing without a central authority [1, 6]; Dynamo extended this toward high availability under unstable network conditions [7]. Membership and failure detection are handled by gossip protocols such as SWIM [8], while mDNS and DNS-SD serve smaller local networks without a dedicated server [10, 11]. The benefit is resilience to single-node failure. The cost is weaker global consistency because peer knowledge can drift apart.

2.1.3 Traffic-Management and Policy-Enforcement Approaches

A third family sits above discovery and focuses on selecting between instances and controlling which connections are allowed. Istio applies load-balancing policies such as round-robin, weighted routing, and circuit breaking through sidecar proxies [14], while Kubernetes NetworkPolicy restricts ingress and egress at the pod level [13]. These tools offer detailed control, but they also add extra infrastructure and split discovery, routing, and policy across multiple layers.

2.2 Security, Reachability, and Policy Correctness

Discovery is only useful if the returned endpoints are actually reachable. Work on firewall policy analysis—detecting rule anomalies [17] and verifying intended behaviour [18]—reinforces that a discovery system must account for the active security policy, not just record which services exist.

2.3 How TDS Differs, and Its Strengths and Weaknesses

Existing solutions usually require an early commitment to one architecture. TDS instead supports both centralised and peer-to-peer operation behind a single client-facing proxy interface. Services issue the same REGISTER and QUERY messages regardless of which backend is active, so the deployment topology can change without changing application code.

This reduces the risk of being tied to one architecture and lowers adoption cost, while still allowing each mode to play to its strengths—centralised for simplicity and control, P2P for resilience and removal of a central dependency. The trade-off is extra implementation complexity, and the limits of each architecture still remain: the centralised mode still depends on a registry server, and the P2P mode is naturally weaker in global consistency. TDS exposes both and lets the operator choose rather than hiding those trade-offs.

3 Design

3.1 Services, Tasks, and the Client Proxy

The diagram below gives an overview of a client machine in this system, with multiple services registering with and querying the client proxy, which is the local endpoint of this solution.

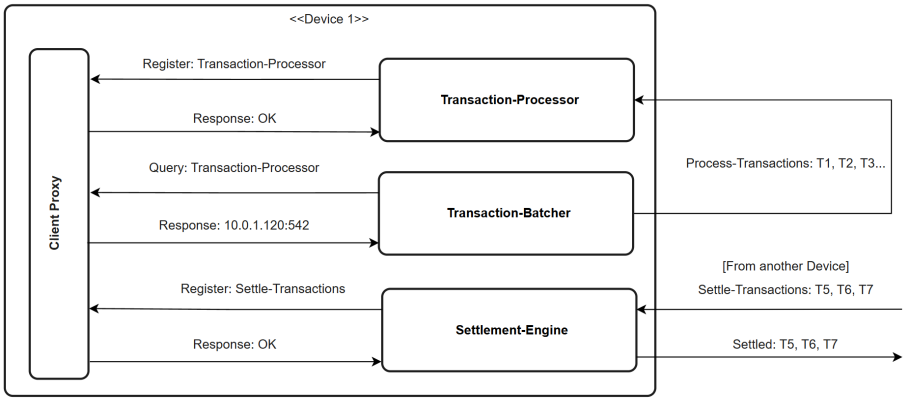

Figure 1: Client machine service registration and discovery via client proxy

In this architecture, services running on a machine query their local instance of the client proxy. From the service's point of view, that is all that is needed to advertise the service, apart from sending periodic heartbeats. The client proxy also handles requests for where those services should send data after finishing their own work. The client proxy is the interface behind which the rest of the project operates.

3.1.1 Message Format

All communication between local services (clients) and the client proxy uses JSON messages in both centralised and P2P modes. The two shared commands are `REGISTER` and `QUERY`.

JSON requests (service → client proxy)

```
{"cmd":"REGISTER","task":"payment","address":"10.0.1.5:8080","capacity":10}
{"cmd":"QUERY","task":"payment"}
```

JSON response format

```
{"status":"OK"} //Register - OK
{"status":"OK","address":"10.0.1.5:8080"} //Successful query
{"status":"NOTFOUND"} // Queried task not found
{"status":"ERR","error":"task required"} // Example error: no task in query
    request
```

These message types are intentionally minimal so the same client-side behaviour works regardless of whether the proxy is currently forwarding to a central server or to the P2P registry.

3.2 Centralised Mode

We now move beyond the scope of a single machine registering and querying services to a distributed system containing many such machines. The figure below shows the role the server plays in centralised mode.

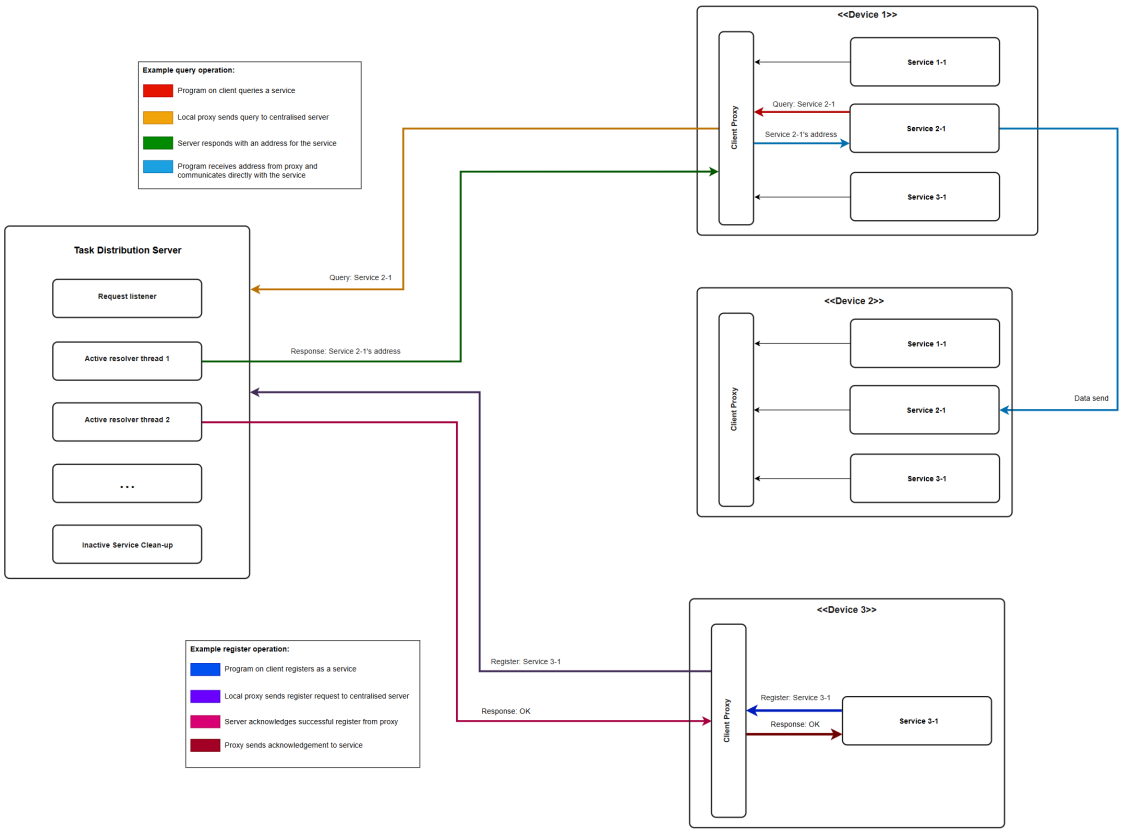

Figure 2: Client machines operating in server architecture

3.2.1 Load Balancing

The server load balances services using round-robin across all services under a task [5]. This spreads traffic evenly. On top of this, a weighted round-robin option uses each registered service's capacity as its weight. This helps prevent services on less powerful machines from receiving the same volume of traffic as stronger ones. During each heartbeat, a service advertises its current capacity so the load balancer can take it into account.

3.2.2 Hybrid Storage: Cache and Persistence

The centralised server uses a hybrid storage strategy that balances performance with durability. At the core is an in-memory cache that holds a subset of frequently queried tasks. On a cache miss, or at regular intervals, the server refreshes the cache by querying a persistent PostgreSQL backend. This gives low-latency responses for hot tasks while keeping durable state for recovery and continued operation.

In practice, the server keeps a configurable cache size and uses query frequency to decide which tasks stay cached. After query-count thresholds or time-based intervals, the cache is refreshed from the database. This stops rarely accessed tasks from using too much memory and makes sure heartbeat updates stored in the database appear when the cache is next refreshed. This hybrid approach was important for achieving good query latency without losing the option to scale through a database backend and survive restarts.

3.2.3 Firewall Mode

In many real uses of this project, all machines can query and register with TDS, but not all services are allowed to communicate with each other. This is enforced by network rules created and managed by a central authority in the system. The purpose of firewall mode is to check a service query against the list of destinations that the requester is allowed to reach. Instead of returning any healthy instance for a task, the server first extracts the requester's IP address, compares it with the configured routing policy, and then filters the candidate service addresses before load balancing among the remaining allowed endpoints. This means a client only receives an address that should be reachable for that source according to the imported policy.

The firewall model implemented in TDS is deliberately simple: it is an allow-list over source and destination IPs. Each rule has two fields, a source and a destination, and each field may be either a single IP address or a CIDR block. This supports both exact host-to-host rules such as `192.168.1.10 10.0.0.5` and broader subnet rules such as `192.168.1.0/24 10.0.0.0/24`. In file-backed mode, rules are stored one per line, blank lines are ignored, and lines beginning with `#` are treated as comments. The rule language does not include ports, protocols, priorities, or deny entries; it is intended only to answer the discovery question "which of these registered service IPs may this client contact?"

This simplified design fits the role of TDS. The discovery server is not replacing the network firewall, but using a compact copy of routing intent to avoid returning unusable endpoints. When service addresses are stored as `ip:port`, the implementation strips the port, evaluates the destination IP against the rule set, and preserves the original address only if the IP is permitted. If no firewall rules are loaded, the mode becomes permissive and all destinations are treated as reachable.

3.3 Peer-to-Peer Mode

The diagram below gives an overview of the peer-to-peer architecture. It shows how tasks are registered and then stored on the correct node in the network. It also shows service queries and the separate sending of data outside the architecture for processing. In contrast to the centralised mode, all functionality in this architecture runs from the client proxy. The P2P network begins with one "bootstrap" node. All later nodes join through an existing node in the network. When a node joins, it hashes its IP address to produce its node ID. This ID is capped at 256 bits, forming a ring network.

3.3.1 Distributed Hash Table (DHT)

The DHT stores task:address mappings in this architecture. When a service registers a task with its client proxy, the proxy hashes the task name to produce a 256-bit key on the same ring used for node IDs. Each node also has a position on that ring, derived by hashing its own address. The proxy then finds the first node whose ID is greater than or equal to the task hash. If the hash falls beyond the largest node ID on the ring, the search wraps around to the smallest node ID. That node is treated as the node "closest" to the task hash and therefore responsible for the key. Once it has identified that node, the client proxy sends it a STORE request with the service details. The node that receives this message performs the same calculation against its own peer list to confirm that it is the best candidate. If so, it stores the task in memory, similar to the cache in centralised mode.

A query follows the same path: the client proxy hashes the requested task, finds the same node whose ID is greater than or equal to the task hash, and sends the lookup there. If that node has the service stored, it returns an address; otherwise it returns NOTFOUND.

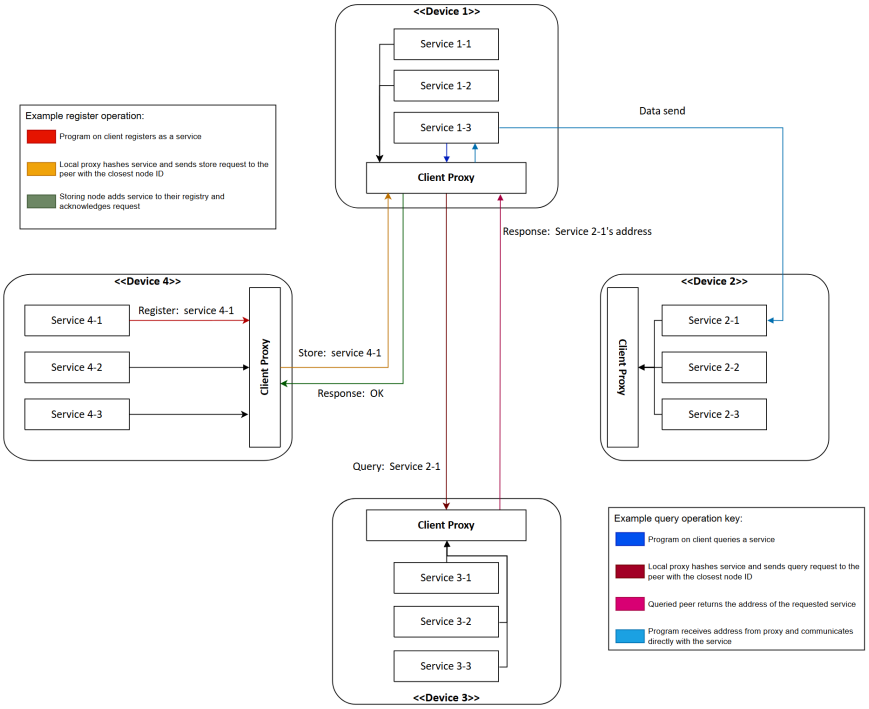

Figure 3: Client machines operating in P2P architecture

3.3.2 Eventual Consistency and Node Discovery

The Distributed Hash Table is eventually consistent by nature. Nodes receive an initial peer list from their bootstrap node. As more nodes join the network, they discover one another through store and query messages. If a node does not already know the requester, it adds that node to its peer list. To support this, a periodic rediscovery process also runs. Nodes query their original bootstrap node for peers that joined after them. This adds extra work to bootstrap nodes, but it can be managed by choosing more powerful devices for that role.

4 Implementation

The following section details the implementation of the project. It includes the structure of the codebase, database schema and key algorithms. Following this it highlights key challenges and optimisations that were made during the development of the project.

4.1 Tools and Technologies

TDS was implemented primarily in Go (Go 1.24) [4]. The main server, client proxy, registry logic, transport handlers, and DHT components are all built from the same Go codebase.

The system uses both in-memory data structures and PostgreSQL, allowing fast lookup for active services while retaining persistent storage where needed. Communication is built around UDP and TCP, with optional TLS and mutual TLS for the centralised mode, and JSON is used for service messages.

For operation and deployment, the project includes a terminal dashboard built with `tview/tcell`, Docker and Kubernetes assets, and supporting PowerShell and shell scripts.

4.2 Codebase Structure

The codebase is organised to separate executable entry points, reusable packages, deployment assets, and system-level test environments. This allowed me to develop centralised and P2P features in parallel while keeping the service-facing interface stable.

At the top level, `cmd/` contains runnable programs. The most important binaries are `cmd/server` for centralised operation and `cmd/client_proxy` for the local proxy process used by services in both architectures. Additional command folders such as `cmd/centralised_demo`, `cmd/client_demo`, `cmd/test_client`, and `cmd/test_json_client` were used for integration validation and protocol probing during development.

Core implementation logic is located in `pkg/`, with package boundaries aligned to architectural concerns:

- `pkg/registry`: task-to-service registration, lookup, heartbeat handling, and load-balancing state.
- `pkg/store`: persistence and cache support for the centralised server path.
- `pkg/transport`: UDP/TCP communication handlers and message routing.
- `pkg/client`: service-facing helper functions used by applications to call REGISTER and QUERY.
- `pkg/dht`: peer management, key placement, and DHT operations for P2P mode.
- `pkg/firewall`: routing-rule filtering used by firewall mode in centralised deployments.

Infrastructure and reproducibility assets are versioned alongside the source. Kubernetes manifests are grouped in `k8s/`, container definitions are under `cmd/*/Dockerfile`, and supporting scripts are in `scripts/`. End-to-end scenario testing is concentrated in `test_scripts/` and `test_environment/`, while the transport simulation used in evaluation is provided in `Simulation/`. This structure made it possible to validate behaviour from unit-level package logic through to multi-process deployment scenarios using the same codebase.

4.3 Database Schema

The centralised mode persists registry state in PostgreSQL using a single primary table, `services`. The design deliberately keeps the relational model compact: one row represents one service instance for one task, keyed by `(task, address)`. This supports fast upsert-style registration and straightforward query aggregation while avoiding joins on the lookup path.

The effective schema used by runtime migrations is shown below.

```
CREATE TABLE IF NOT EXISTS services (
    id SERIAL PRIMARY KEY,
    task TEXT NOT NULL,
    address TEXT NOT NULL,
    last_heartbeat TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
    query_count BIGINT NOT NULL DEFAULT 0,
    capacity INTEGER NOT NULL DEFAULT 1,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
    UNIQUE(task, address)
);
```

This schema is accompanied by indexes on `task`, `last_heartbeat`, and `is_active`. Together these support the three dominant database operations in the system: task-based lookups, heartbeat timeout cleanup, and filtering of active versus inactive rows.

Two implementation decisions are important for system behaviour:

- **Idempotent registration via upsert.** Register operations use `INSERT ... ON CONFLICT (task, address) DO UPDATE` so heartbeats and capacity updates refresh existing rows without duplicate records.

- **Soft cleanup.** Expired services are marked `is_active = FALSE` rather than deleted. This preserves historical rows for observability and audit-style analysis while keeping active-query paths efficient. If a service comes back later, its existing row is simply reactivated instead of creating a new one, so the database does not grow much from repeated churn.

The `query_count` and `capacity` columns support weighted load balancing and usage analytics. In practice, this allows the server to combine performance-oriented in-memory selection with periodic or direct persistence of usage counters, maintaining both runtime speed and durable operational state.

4.4 Functional Testing

Comprehensive functional testing was carried out across both centralised and P2P modes, covering core operations such as service registration, heartbeat handling, query routing, load balancing, and cleanup semantics. Test coverage includes unit tests for individual packages, integration tests for multi-component scenarios, and property-based testing for protocol correctness. Detailed test results and harnesses are provided in `test_scripts/` and `test_environment/` within the repository.

4.5 Integration Testing: Realistic Application Scenario

To validate practical usefulness, I implemented a simulation based on a highly simplified version of the TfL Oyster, payment card, and physical ticket system. This simulation includes station computers, payment gates, fare calculation services, and multiple ticket distribution endpoints, all connected through TDS service discovery. The system showed smooth communication across components, with the only troubleshooting required being the initial server startup. This supported the robustness of both the registry logic and the transport mechanisms. The complete system architecture and component interactions are documented in the High-Level Design document in the repository.

5 Technical Highlights and Optimisations

This section highlights parts of the project that were especially challenging. It explains how I addressed them and where I chose to stop further work within the project timeframe.

5.1 Cache and Concurrency Performance

The hybrid cache-plus-database design created an unexpected performance bottleneck. During testing, I found that cache *misses* were actually faster than *hits*. This surprising result exposed a concurrency problem: although queries should mostly be reads, every query also updated round-robin state and query counters. Both of these are write operations that require mutual exclusion. Even on the cache path, which should be the fastest case, readers were competing for a write lock held during those updates, causing heavy contention.

The solution involved two complementary optimisations:

5.1.1 Batched Query Count Updates

Query counts were used only to decide which tasks should be cached, not for client-facing operations. That made it safe to batch counter updates asynchronously. Instead of writing to the database on every query, counters are collected in memory and flushed only before cache refresh intervals. This removed one of the two write operations from the hot query path.

5.1.2 Lock-Free Round-Robin with Atomics

The round-robin load-balancing index still needed to be updated on every query to track which service should be selected next for each task. Replacing the mutex-protected counter with atomic integers

allowed lock-free, wait-free updates to this state. Atomic operations in Go (via `atomic.Int64`) use CPU-level compare-and-swap instructions and do not require OS-level mutual exclusion. For a high-frequency operation such as updating the round-robin index thousands of times per second, this removes the overhead of lock acquisition, contention detection, and context switching. As a result, incrementing the atomic counter is far faster than taking a mutex, updating the value, and releasing the lock.

With both optimisations in place, the cache became measurably faster than the database backend, which matches the expected result that cache hits should outperform cache misses. This showed how subtle concurrency bottlenecks can appear even in simple designs and highlighted the importance of profiling when deciding where lock-free atomics and asynchronous batching are most useful.

5.1.3 Peer-to-Peer Challenges

During testing, a routing inconsistency became visible as the network grew. The DHT initially assigned each task to one responsible node using `FindClosestNode`, which performs a binary search on a sorted ring of hashed node IDs.

The problem came from the way nodes build their peer lists through gossip: a node only learns about a peer when it is contacted directly. In larger networks, less active nodes were contacted less often, so their peer lists became outdated. As a result, two nodes with different peer views could disagree on which node was closest to a given task hash.

A service could register on node A, which believed it was responsible, while a query arriving at node B could calculate a different "closest" node and either forward the query to the wrong place or return an empty result even though the data existed in the network. This inconsistency appeared during multi-node integration tests: queries that should have succeeded would sometimes return nothing, and the failure rate increased with network size.

To address this, single-node responsibility was replaced with *k-nearest-neighbour replication*. The `FindKClosestNodes` function computes the k nodes with the smallest ring distance to a task hash, and registrations are written to all k of them.

Lookups first check local storage, then check whether the querying node is among the k nearest nodes. If it is not, it forwards the query to each of the k responsible nodes in turn and returns the first non-empty response (`forwardLookup`). Because the data is replicated across k nodes, a query only fails if all k copies are absent or unreachable at the same time.

This made the system tolerant of the routing-table divergence that caused the original inconsistency. Even if one node's peer list is stale and it targets the wrong primary, one of the remaining $k-1$ replicas should still respond correctly. The trade-off is k-fold write amplification, but for service discovery this is acceptable because registrations are much less frequent than queries, and heartbeats keep the replicated entries fresh. The network therefore gains robustness against node failure and stale peer knowledge at the cost of higher write overhead. At large enough scales, the centralised approach remains more predictable, which is why TDS keeps both modes available.

5.1.4 Firewall Mode

Firewall mode was implemented only for the centralised server path, where each query already passes through a single decision point. The server can be configured with either a file-backed or database-backed firewall rule source. In both cases, the runtime result is the same: the rules are loaded into an in-memory firewall structure and query responses are filtered before the usual weighted round-robin selection is applied.

For file-backed loading, the server reads a plaintext rules file during startup. The parser scans line-by-line, discards empty lines and comments, splits each non-empty line into exactly two whitespace-separated fields, and then parses each field as either a single IP address or a CIDR network. Any malformed line causes startup to fail with a descriptive error including the line number. Once loaded, the rules are stored as parsed IP or network objects so that subsequent query-time checks only perform address matching rather than reparsing text.

For database-backed loading, the server uses a rule-provider abstraction and reads from a dedicated

`firewall_rules` table. The schema separates exact-host rules from subnet rules by storing nullable columns for both forms of each endpoint, which keeps the representation simple while still supporting either syntax:

```
CREATE TABLE IF NOT EXISTS firewall_rules (
    id SERIAL PRIMARY KEY,
    source_ip TEXT,
    source_network TEXT,
    dest_ip TEXT,
    dest_network TEXT,
    created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_firewall_rules_source
    ON firewall_rules(source_ip, source_network);
```

At load time, each row is converted back into the same internal rule structure used by the file parser. If `source_network` or `dest_network` is present it is parsed as CIDR notation; otherwise the corresponding `source_ip` or `dest_ip` field is parsed as a single host address. This means both storage backends converge on the same matching logic and the registry code does not need to care where the rule set originated. The main limitation is architectural rather than technical: this approach depends on TDS being supplied with a faithful copy of firewall intent, so consistency with the real network policy still depends on an external process keeping the file or database entries up to date.

6 Conclusion and Discussion

6.1 Current Weaknesses and Development Points

The main remaining weaknesses in TDS are not in its core ability to register and resolve services, but in the maturity and precision of some surrounding features. The most important development points are:

Health checking: The current design depends mainly on heartbeats to identify stale services. This is simple and effective, but it means TDS only knows a service has failed once heartbeats stop arriving. Active health probing would allow failed instances to be detected and removed more quickly.

Firewall efficiency: Firewall-aware routing is functionally useful, but it currently adds extra matching work on the query path. This could be improved by optimising rule lookup and filtering so that firewall mode scales more efficiently under larger rule sets and higher query volumes.

Finer-grained firewall control: The firewall model currently focuses on host and network-level reachability. A more complete implementation would support finer-grained policy decisions, particularly port-level rules, so that TDS can reflect real deployment constraints more accurately.

P2P scalability: The peer-to-peer mode works well at the scale explored in this project, but its peer-discovery approach would need refinement for larger deployments. More structured peer membership and routing maintenance would improve scalability without changing the client-facing workflow.

Operational observability: The terminal dashboard is useful for development and demonstration, but operational visibility could be improved further with richer exported metrics and more detailed tracing of routing and filtering decisions.

6.2 Conclusion

This project has met the objectives set out at the start. TDS provides a flexible architecture that balances correctness with decentralisation, and the client proxy acts as a stable, uniform interface to all services on a device. Because the query, registration, and response formats are clearly defined, any service can integrate TDS by replacing a hard-coded endpoint with a single client library call.

Centralised mode is well-suited to deployments with a dedicated discovery server. When memory is plentiful the server can operate entirely in-memory: because all records must be kept fresh via heart-

beats, a restart causes minimal disruption. Pairing the server with a database extends its capacity and allows longer heartbeat timeouts, while a high-performance cache keeps the most-queried entries fast. The firewall feature adds another layer of utility by filtering query responses against imported routing rules, ensuring that only reachable endpoints are returned to a caller. As noted in the implementation discussion, TDS does not aim to replace dedicated network security tooling; the routing is best-effort. In hybrid (database-backed) mode, multiple server instances can share load because both tasks and firewall rules live in the database rather than in local memory. Cache divergence between instances is possible, but because all registrations go directly to the database the effect on task listings is small.

Peer-to-peer mode trades some consistency and extra per-client computation for fault tolerance and the removal of any single point of failure. The k-nearest-neighbour algorithm makes lookups resilient to both peer failure and stale peer knowledge. As discussed in the implementation section, this mode suits smaller deployments where a dedicated discovery server would be wasteful, although its gossip-based peer discovery limits scalability at larger scales. That limitation is manageable because TDS is multimodal: integrators can choose the topology that best fits their environment.

The weighted round-robin load balancer used in both modes deserves a mention. It distributes traffic proportionally, so lightweight devices are not overwhelmed alongside more powerful ones. Combined with firewall-aware routing in centralised mode, it makes TDS straightforward to slot into an existing system. This was evident during the development of the simulation environment: designing the distributed system as a whole was challenging, yet routing data between its components for processing required no special effort at all.

In summary, TDS has exceeded my initial expectations. It has grown into a system that performs its intended role effectively while still leaving clear areas for future refinement. I am proud of what has been built and of the consistency with which it has been developed throughout the year.

References

[1] Stoica, I., Morris, R., Karger, D., Kaashoek, M. F., & Balakrishnan, H. (2001). *Chord: A scalable peer-to-peer lookup service for internet applications*. ACM SIGCOMM Computer Communication Review, 31(4), 149-160.

[2] HashiCorp. (2021). *Consul: Service Mesh for Microservices*. https://www.consul.io/

[3] CoreOS. (2020). *etcd: A distributed, reliable key-value store*. https://etcd.io/

[4] Donovan, A. A., & Kernighan, B. W. (2015). *The Go Programming Language*. Addison-Wesley Professional.

[5] Cardellini, V., Colajanni, M., & Yu, P. S. (1999). *Dynamic load balancing on web-server systems*. IEEE Internet Computing, 3(3), 28-39.

[6] Maymounkov, P., & Mazieres, D. (2002). *Kademlia: A peer-to-peer information system based on the XOR metric*. International Workshop on Peer-to-Peer Systems (IPTPS), 53-65.

[7] DeCandia, G., Hastorun, D., Jampani, M., Kakulapati, G., Lakshman, A., Pilchin, A., Sivasubramanian, S., Vosshall, P., & Vogels, W. (2007). *Dynamo: Amazon's highly available key-value store*. ACM SIGOPS Operating Systems Review, 41(6), 205-220.

[8] Das, A., Gupta, I., & Motivala, A. (2002). *SWIM: Scalable weakly-consistent infection-style process group membership protocol*. IEEE/IFIP International Conference on Dependable Systems and Networks, 303-312.

[9] Ongaro, D., & Ousterhout, J. (2014). *In search of an understandable consensus algorithm (extended version)*. USENIX Annual Technical Conference, 305-319.

[10] Cheshire, S., & Krochmal, M. (2013). *Multicast DNS* (RFC 6762). IETF.

[11] Cheshire, S., & Krochmal, M. (2013). *DNS-Based Service Discovery* (RFC 6763). IETF.

[12] Kubernetes Authors. (2026). *Kubernetes Services, Load Balancing, and Networking: Service*. https://kubernetes.io/docs/concepts/services-networking/service/

[13] Kubernetes Authors. (2024). *Kubernetes Network Policies*. https://kubernetes.io/docs/concepts/services-networking/network-policies/

[14] Istio Authors. (2025). *Traffic Management Concepts*. https://istio.io/latest/docs/concepts/traffic-management/

[15] HashiCorp. (2026). *Consul Discover Services Overview*. https://developer.hashicorp.com/consul/docs/discover

[16] etcd Authors. (2025). *etcd API guarantees*. `https://etcd.io/docs/v3.6/learning/api_guarantees/`

[17] Al-Shaer, E., & Hamed, H. (2009). *Firewall policy queries*. IEEE Transactions on Parallel and Distributed Systems, 20(6), 766-777.

[18] Jeffrey, A., & Samak, T. (2009). *Model checking firewall policy configurations*. IEEE International Symposium on Policies for Distributed Systems and Networks, 60-67.